

Ejercicio 1: Termometro

```
namespace EjerciciosPOO
{
    public class Termometro
    {
        // Atributo privado
        private double _temperaturaCelsius;

        // Constructor
        public Termometro(double temperaturainicial)
        {
            TemperaturaCelsius = temperaturainicial;
        }

        // Propiedad (get y set con validación)
        public double TemperaturaCelsius
        {
            get { return _temperaturaCelsius; }
            set
            {
                if (value >= -273.15)
                    _temperaturaCelsius = value;
                else
                    Console.WriteLine("Error: La temperatura no puede estar por debajo del cero absoluto (-273.15 °C).");
            }
        }

        // Métodos
        public double ObtenerFahrenheit()
        {
            return (_temperaturaCelsius * 9 / 5) + 32;
        }

        public double ObtenerKelvin()
        {
            return _temperaturaCelsius + 273.15;
        }
    }
}

// MAIN
using EjerciciosPOO;

Console.WriteLine("---- Ejercicio 1: Termómetro ----");
Termometro t1 = new Termometro(25); // 25 grados
```

```
Console.WriteLine($"T1: {t1.TemperaturaCelsius}°C es {t1.ObtenerFahrenheit()}°F y {t1.ObtenerKelvin()}K");
```

```
Termometro t2 = new Termometro(-300); // Probará la validación  
t2.TemperaturaCelsius = 0;  
Console.WriteLine($"T2 ajustado: {t2.TemperaturaCelsius}°C es {t2.ObtenerFahrenheit()}°F");
```

Ejercicio 2: Reloj Digital

```
namespace EjerciciosPOO
```

```
{  
    public class Reloj  
    {  
        private int _hora, _minutos, _segundos;  
  
        public Reloj(int hora, int minutos, int segundos)  
        {  
            Hora = hora;  
            Minutos = minutos;  
            Segundos = segundos;  
        }  
  
        public int Hora  
        {  
            get { return _hora; }  
            set { _hora = (value >= 0 && value <= 23) ? value : 0; }  
        }  
  
        public int Minutos  
        {  
            get { return _minutos; }  
            set { _minutos = (value >= 0 && value <= 59) ? value : 0; }  
        }  
  
        public int Segundos  
        {  
            get { return _segundos; }  
            set { _segundos = (value >= 0 && value <= 59) ? value : 0; }  
        }  
  
        public void MostrarHora()  
        {  
            // Formato HH:MM:SS  
            Console.WriteLine($"{_hora:D2}:{_minutos:D2}:{_segundos:D2}");  
        }  
    }  
}
```

```
}
```

```
// MAIN
```

```
using EjerciciosPOO;
```

```
Console.WriteLine("\n---- Ejercicio 2: Reloj Digital ----");
```

```
Reloj reloj1 = new Reloj(14, 30, 15);
```

```
Console.Write("Reloj 1: ");
```

```
reloj1.MostrarHora();
```

```
Reloj reloj2 = new Reloj(25, 61, -5); // Valores inválidos, se setearán a 0
```

```
Console.Write("Reloj 2 (inválido): ");
```

```
reloj2.MostrarHora();
```

Ejercicio 3: Cuenta bancaria simple

```
namespace EjerciciosPOO
```

```
{
```

```
    public class Cuenta
```

```
    {
```

```
        private string _titular;
```

```
        private double _saldo;
```

```
        public Cuenta(string titular, double saldoInicial)
```

```
        {
```

```
            _titular = titular;
```

```
            _saldo = saldoInicial > 0 ? saldoInicial : 0;
```

```
        }
```

```
        public string Titular
```

```
        {
```

```
            get { return _titular; }
```

```
            set { _titular = value; }
```

```
        }
```

[cite_start]// El set es privado para asegurar que solo se modifique por métodos [cite:

16]

```
        public double Saldo
```

```
        {
```

```
            get { return _saldo; }
```

```
            private set { _saldo = value; }
```

```
        }
```

```
        public void Depositar(double cantidad)
```

```
        {
```

```
            if (cantidad > 0)
```

```

        {
            Saldo += cantidad;
            Console.WriteLine($"Depósito exitoso. Nuevo saldo: {Saldo}");
        }
    }

    public void Retirar(double cantidad)
    {
        if (cantidad > 0 && cantidad <= Saldo)
        {
            Saldo -= cantidad;
            Console.WriteLine($"Retiro exitoso. Nuevo saldo: {Saldo}");
        }
        else
        {
            Console.WriteLine("Fondos insuficientes o cantidad inválida.");
        }
    }
}

```

// MAIN

using EjerciciosPOO;

Console.WriteLine("\n---- Ejercicio 3: Cuenta Bancaria ----");

Cuenta cuenta1 = new Cuenta("Ana", 1000);

cuenta1.Depositar(500);

cuenta1.Retirar(200);

Cuenta cuenta2 = new Cuenta("Luis", 500);

cuenta2.Retirar(1000); // Fondos insuficientes

Ejercicio 4: Registro de libros

namespace EjerciciosPOO

{

public class Libro

{

private string _titulo;

private string _autor;

private string _isbn;

private int _numeroPaginas;

public Libro(string titulo, string autor, string isbn, int numeroPaginas)

{

_titulo = titulo;

_autor = autor;

```

        _isbn = isbn;
        _numeroPaginas = numeroPaginas;
    }

    public int NumeroPaginas
    {
        get { return _numeroPaginas; }
        set { _numeroPaginas = value; }
    }

    public bool EsLargo()
    {
        return _numeroPaginas > 300;
    }
}

// MAIN
using EjerciciosPOO;

Console.WriteLine("\n---- Ejercicio 4: Registro de Libros ----");
Libro libro1 = new Libro("El Quijote", "Cervantes", "12345", 800);
Console.WriteLine($"¿'{libro1.NumeroPaginas}' páginas es largo?: {libro1.EsLargo()}");

Libro libro2 = new Libro("El Principito", "Saint-Exupéry", "67890", 96);
Console.WriteLine($"¿'El Principito' es largo?: {libro2.EsLargo()}");

```

Ejercicio 5: Control de inventario

```

namespace EjerciciosPOO
{
    public class Producto
    {
        private string _codigo;
        private string _nombre;
        private double _precio;

        public Producto(string codigo, string nombre, double precio)
        {
            _codigo = codigo;
            _nombre = nombre;
            _precio = precio;
        }

        public double Precio
        {
            get { return _precio; }
        }
    }
}

```

```

        set { _precio = value; }
    }

    public void AplicarDescuento(double porcentaje)
    {
        if (porcentaje > 0 && porcentaje <= 100)
        {
            double descuento = _precio * (porcentaje / 100);
            _precio -= descuento;
            Console.WriteLine($"Descuento aplicado. Nuevo precio: {_precio}");
        }
    }
}
}
}

```

// MAIN

using EjerciciosPOO;

Console.WriteLine("\n---- Ejercicio 5: Control de Inventario ----");

Producto p1 = new Producto("A01", "Laptop", 1000);

p1.AplicarDescuento(10); // Queda en 900

Producto p2 = new Producto("A02", "Mouse", 50);

p2.AplicarDescuento(50); // Queda en 25

Ejercicio 6: Seguimiento de Fitness

namespace EjerciciosPOO

```

{
    public class CalculadoraPasos
    {
        private string _nombreUsuario;
        private int _metaDiaria;
        private int _pasosAcumulados;

        public CalculadoraPasos(string nombreUsuario, int metaDiaria)
        {
            _nombreUsuario = nombreUsuario;
            _metaDiaria = metaDiaria;
            _pasosAcumulados = 0; // Inicia en 0
        }

        public void RegistrarPasos(int pasos)
        {
            if (pasos > 0)
                _pasosAcumulados += pasos;
        }
    }
}

```

```

        public bool ObjetivoCumplido()
        {
            return _pasosAcumulados >= _metaDiaria;
        }
    }
}

```

// MAIN

using EjerciciosPOO;

```

Console.WriteLine("\n---- Ejercicio 6: Seguimiento Fitness ----");
CalculadoraPasos calc1 = new CalculadoraPasos("Carlos", 10000);
calc1.RegistrarPasos(5000);
calc1.RegistrarPasos(6000);
Console.WriteLine($"¿Cumplió su meta Carlos?: {calc1.ObjetivoCumplido()}");

```

```

CalculadoraPasos calc2 = new CalculadoraPasos("María", 8000);
calc2.RegistrarPasos(3000);
Console.WriteLine($"¿Cumplió su meta María?: {calc2.ObjetivoCumplido()}");

```

Ejercicio 7: rectangulo prohibido

namespace EjerciciosPOO

```

{
    public class Rectangulo
    {
        private double _ancho;
        private double _alto;

        public Rectangulo(double ancho, double alto)
        {
            if (ancho < 0 || alto < 0)
            {
                Console.WriteLine("Error: Valores negativos no permitidos. Se asignó 0.");
                _ancho = 0;
                _alto = 0;
            }
            else
            {
                _ancho = ancho;
                _alto = alto;
            }
        }

        public double Area() { return _ancho * _alto; }
        public double Perimetro() { return 2 * (_ancho + _alto); }
    }
}

```

```

    }
}

// MAIN
using EjerciciosPOO;

Console.WriteLine("\n---- Ejercicio 7: Rectángulo ----");
Rectangulo r1 = new Rectangulo(5, 10);
Console.WriteLine($"R1 - Área: {r1.Area()}, Perímetro: {r1.Perimetro()}");

Rectangulo r2 = new Rectangulo(-3, 8); // Constructor lanza mensaje
Console.WriteLine($"R2 (inválido) - Área: {r2.Area()}");

```

Ejercicio 8: Perfil de Usuario

```

namespace EjerciciosPOO
{
    public class Usuario
    {
        private string _nombreDeUsuario;
        private string _password;
        private string _email;

        public Usuario(string nombreDeUsuario, string password, string email)
        {
            _nombreDeUsuario = nombreDeUsuario;
            _password = password;
            _email = email;
        }

        public void CambiarPassword(string antigua, string nueva)
        {
            if (antigua == _password)
            {
                _password = nueva;
                Console.WriteLine("Contraseña actualizada con éxito.");
            }
            else
            {
                Console.WriteLine("Error: La contraseña antigua no coincide.");
            }
        }
    }
}

// MAIN
using EjerciciosPOO;

```



```

Console.WriteLine("\n---- Ejercicio 8: Perfil de Usuario ----");
Usuario user1 = new Usuario("user123", "clave123", "user@email.com");
user1.CambiarPassword("claveIncorrecta", "nuevaClave");
user1.CambiarPassword("clave123", "nuevaClave");

Usuario user2 = new Usuario("admin", "adminPass", "admin@email.com");
user2.CambiarPassword("adminPass", "secure123");

```

Ejercicio 9: Cafetera automatica

```

namespace EjerciciosPOO
{
    public class Cafetera
    {
        private int _capacidadMaxima;
        private int _cantidadActual;

        public Cafetera(int capacidadMaxima, int cantidadActual)
        {
            _capacidadMaxima = capacidadMaxima;
            _cantidadActual = cantidadActual > capacidadMaxima ? capacidadMaxima :
cantidadActual;
        }

        public void LlenarCafetera()
        {
            _cantidadActual = _capacidadMaxima;
            Console.WriteLine("Cafetera llena.");
        }

        public void ServirTaza(int cantidad)
        {
            if (cantidad <= _cantidadActual)
            {
                _cantidadActual -= cantidad;
                Console.WriteLine($"Se sirvió la taza. Queda: {_cantidadActual}");
            }
            else
            {
                Console.WriteLine($"No hay café suficiente. Se sirvió lo que quedaba
({_cantidadActual})");
                _cantidadActual = 0;
            }
        }

        public void VaciarCafetera()

```

```

        {
            _cantidadActual = 0;
            Console.WriteLine("Cafetera vaciada.");
        }
    }
}

```

// MAIN

```
using EjerciciosPOO;
```

```
Console.WriteLine("\n---- Ejercicio 9: Cafetera ----");
```

```
Cafetera caf1 = new Cafetera(1000, 500);
```

```
caf1.ServirTaza(200);
```

```
caf1.LlenarCafetera();
```

```
Cafetera caf2 = new Cafetera(500, 100);
```

```
caf2.ServirTaza(200); // Vaciara lo que quede
```

Ejercicio 10: Promedio escolar

```
namespace EjerciciosPOO
```

```
{
```

```
    public class Estudiante
```

```
    {
```

```
        private string _nombre;
```

```
        private double[] _notas;
```

```
        public Estudiante(string nombre, double nota1, double nota2, double nota3)
```

```
        {
```

```
            _nombre = nombre;
```

```
            _notas = new double[] { nota1, nota2, nota3 }; // Arreglo de 3 notas [cite: 31]
```

```
        }
```

```
        public double CalcularPromedio()
```

```
        {
```

```
            double suma = 0;
```

```
            foreach (double nota in _notas)
```

```
            {
```

```
                suma += nota;
```

```
            }
```

```
            return suma / _notas.Length;
```

```
        }
```

```
        public string EstadoFinal()
```

```
        {
```

```
            return CalcularPromedio() >= 6 ? "Aprobado" : "Reprobado";
```

```
        }
```

```
}  
}
```

```
// MAIN
```

```
using EjerciciosPOO;
```

```
Console.WriteLine("\n---- Ejercicio 10: Promedio Escolar ----");
```

```
Estudiante est1 = new Estudiante("Laura", 7, 8, 6);
```

```
Console.WriteLine($"Promedio de Laura: {est1.CalcularPromedio()} - Estado:  
{est1.EstadoFinal()}");
```

```
Estudiante est2 = new Estudiante("Pedro", 4, 5, 5);
```

```
Console.WriteLine($"Promedio de Pedro: {est2.CalcularPromedio():F2} - Estado:  
{est2.EstadoFinal()}");
```